

TP Kubernetes, SERVICES MESH sous MiniKube

Étape 1 : manipulations

Objectifs :

Après avoir installé un environnement conforme kubernetes avec le bundle MINIKUBE, voir le document « [document tp_kubernetes.pdf](#) », vous présenterez dans un premier temps, dans un document PDF ou Markdown (*) les premiers tests (Questions Kubernetes ci dessous) de l'environnement docker et Kubernetes en répondant aux questions qui suivent.

()Vous mettez la ou les commande(s) utilisée(s) et une preuve de bon fonctionnement de celle ci (capture écran de la CLI par exemple) pour chacune des questions ci dessous.*

Questions Kubernetes.

Démarrer **minikube** pour pouvoir travailler sur **kubernetes** avec **kubectl**

Gestion de minikube

- (1) Vérifiez que minikube pointe correctement vers le moteur docker ?
- (2) Quels sont les addons actuellement installé ?
- (3) Installez celle qui vous semble intéressante, pourquoi ?
- (4) Lister les profils actifs sous minikube avec toutes ses caractéristiques ?
- (5) Quels sont les profils en cours ?
- (6) Comment puis je créer un nouveau profil, que représente un profil ?
- (7) Afficher le statut de minikube ?
- (8) Comment puis je accéder au dashboard de minikube ?
- (9) Qu'est ce que le Dashboard, que présente t'il ?
- (10) Lister les nœuds d'un profil ?
- (11) Ajouter un nœud à un profil minikube, supprimer ce même nœud.
- (12) Consulter les logs de minikube, comment faire ?

Gestion des pods et services sous kubernetes

- (13) Lister les images actuellement en exécution dans votre environnement minikube
- (14) Lancer une image nginx dans un pod et/ou un deployment en mode impératif.
- (15) Créer un service en mode impératif permettant d'accéder à votre service nginx.
- (16) Visualiser les informations du pod et du service.
- (17) Obtenir l'url du service.
- (18) Exécuter le service dans lynx ou Firefox ou tout autre browser.
- (19) Comment puis je lancer une commande bash directement dans mon conteneur nginx ?
- (20) Lister les logs du conteneur nginx du pod ?

Arrêter Minikube

Questions Docker Build

Préparations :

Créez un web service REST sous java spring boot qui répond en mode

- GET au service `/monservice/echo $nom`

- POST au service `/monservice/hello (body en json {"nom":value})`

Peut importe le numéro de port choisi !

Ce service doit être simple et ne servir que de démonstrateur, ne chercher donc pas à faire un service REST complexe ici.

Cas 1 :

Compilez ce service web REST en une archive **fat-Jar(*)**, cette archive devra être complète et contenir toutes les dépendances Spring Boot nécessaires. (Aider vous des plugins Maven pour cela).

Une fois ce service compilé et testé en local. (test non conteneurisé pour preuve de bon fonctionnement).

(*)Fat-jar ?:

- ➔ <https://stackoverflow.com/questions/16222748/building-a-fat-jar-using-maven>
- ➔ <https://imagej.net/develop/uber-jars#:~:text=An%20uber%2DJAR%E2%80%94also%20known,needing%20any%20other%20Java%20code.>

Vous créerez un fichier **dockerfile** qui permettra d'exécuter ce fat-jar directement dans un conteneur docker.

Vous "builderez" ce conteneur et déploierez celui ci a partir d'un fichier **docker-compose**, en preuve de fonctionnement vous testerez a nouveau ce service via l'accès au container en cour d'exécution.

Vous allez refaire le même "buid", mais cette fois ci sans compilation préalable en local.

Vous utiliserez ici la technique du build multi stage, qui consiste à compiler vos sources a partir d'un **dockerfile multiStage**, vous permettra de générer une première image et d'utiliser cette image intermédiaire afin de construire une image docker définitive, ceci en une seule passe.

Exemples & documentations :

- ➔ <https://docs.docker.com/build/building/multi-stage/>
- ➔ <https://medium.com/@office.yeon/dockerizing-with-multi-stage-builds-in-spring-boot-multi-module-project-1fd3aa886afc>
- ➔ <https://medium.com/@office.yeon/dockerizing-with-multi-stage-builds-in-spring-boot-multi-module-project-1fd3aa886afc>

Quel est l'intérêt de cette technique ?

Vous présenterez pour chacune des techniques ci dessus les fichiers **dockerfile** et **docker-compose** !

Étape 2 : Développement d'un projet d'une architecture de type SERVICE MESH

☺ “Mini-Système de Suivi des Analyses de Laboratoire (LaboTrack)”

Un petit système qui simule le parcours d'un échantillon biologique dans un laboratoire d'analyses médicales, en utilisant 3 micro-services maximum, très simples, mais suffisamment réalistes pour être parlants.

Le projet « LaboTrack » consiste à concevoir un mini-système d'information de laboratoire (LIS) simplifié, destiné à simuler de manière réaliste le cycle de vie d'un échantillon biologique au sein d'un laboratoire d'analyses médicales.

Bien que volontairement limité à 3 micro-services, ce système reproduit les étapes clés du cheminement d'un prélèvement, depuis sa réception jusqu'à la validation du résultat.

L'objectif est de proposer une architecture légère mais représentative, permettant d'illustrer :

la gestion des flux de données entre micro-services en service Mesh,
la traçabilité d'un échantillon,
la modélisation des étapes métier d'un laboratoire,
la mise en place d'événements ou messages techniques simples via gRPC,
l'observation d'un processus end-to-end en environnement distribué.

Objectifs opérationnels du projet

« LaboTrack » vise à :

- Simuler le workflow complet d'un échantillon :
Enregistrement → Pré-analyse → Analyse → Validation → Restitution.

- ▶ Mettre en œuvre une architecture micro-services minimaliste, facile à comprendre et à déployer.
- ▶ Illustrer les échanges interservices (gRPC), pour montrer comment un système réparti peut gérer un processus métier selon une architecture (Services Mesh).
- ▶ Être facilement extensible, afin de pouvoir introduire des modules supplémentaires si nécessaire (ex : stockage, supervision, tableau de bord...).

Architecture fonctionnelle simplifiée

Le système repose sur 3 micro-services indépendants, chacun représentant une étape clé :

1) Service "Réception / Enregistrement"

Enregistre un échantillon dès son arrivée.

Attribue un identifiant unique.

Contrôle les données de base : patient, examen demandé, type d'échantillon.

Envoie un événement ou une requête au service suivant.

2) Service "Analyse / Traitement"

Simule la phase d'analyse technique de l'échantillon.

Applique une logique simple : génération d'un résultat fictif (valeur, unité, interprétation).

Peut intégrer un temps de traitement simulé.

Transmet ensuite l'examen au service de validation.

3) Service "Validation / Retour de Résultat" (optionnel selon la simplification souhaitée)

Vérifie la cohérence du résultat produit.

Applique une "signature" biologique (preuve de validation simulée).

Met le résultat final à disposition (API, stockage simple, fichier JSON...).

=> Cycle de vie type d'un échantillon <=

Création de la demande

Un client ou technicien appelle l'API du service de réception.

Transmission au service d'analyse

Un message est envoyé pour déclencher la phase de traitement.

Génération d'un résultat

Valeur numérique, interprétation automatique, horodatage.

Validation biologique simulée

Le service appose une validation ou renvoie une anomalie fictive.

Restitution / Consultation

Le système expose une API pour visualiser l'état d'avancement ou le résultat final.

Objectifs techniques du projet

- ▶ Découvrir le découpage en micro-services via un cas réel du secteur santé.
- ▶ Manipuler Minikube + Kubernetes.
- ▶ Mettre en œuvre un Service Mesh Linkerd sur un cas concret :
 - mTLS automatique
 - Monitoring (stat, tap, top)
 - ServiceProfile (timeouts / retries / idempotence)
 - Zero-Trust interne via ServerAuthorization

Description du projet "LaboTrack"

Le projet simule la chaîne pré-post-analytique d'un examen médical, réduite à l'essentiel.

Micro-Service N°1 – “sample-api”

Rôle : gérer la “réception” d'un échantillon.

Endpoint :

- `POST /samples`` : enregistre un échantillon fictif (stockage BDD), retourne un numéro d'échantillon {id}.
- `GET /samples/{id}`` : retourne l'état actuel.

Micro-Service N°2 – “analysis-api”

Rôle : simuler le traitement d'un échantillon.

Endpoint :

- `POST /analyze/{id}`` : lance une analyse, retourne un résultat (aléatoire, ex. glycémie=0.86 g/L).
- Ajout possible : latence artificielle (`sleep 300 ms``) pour démontrer les « retries ».

Micro-Service N°3 – “result-frontend”

Rôle : afficher les résultats dans une petite page HTML dynamique.

- Cette page consommera les api `sample-api`` et `analysis-api`` pour créer une vue simple.

Comment ces micro-services interagissent ?

1. L'utilisateur envoie un `POST /samples``, reçoit un ID.
2. Il appelle `POST /analyze/{id}``.
3. Le backend appelle directement `sample-api`` pour récupérer les infos (appel inter-service).
4. Le résultat est renvoyé au frontend.

Todo

Construire pour chacun de vos modules un fichier "**dockerfile multistage**", permettant de "**builder**" chaque module en une seule commande de build avec docker, idéalement permettre en une seule commande de compiler l'ensemble des sources et construire des images compatible "OCI"(*) pour un déploiement docker et surtout à destination d'un déploiement "kubernetes".

(*)<https://github.com/opencontainers/image-spec/blob/main/spec.md>

Vous proposerez une solution afin de pousser les images vers une "**registry**" docker en local (voir déploiement d'une "registry" docker locale) ou distance publique (dockerhub par exemple).

Exemple : <https://www.mytinydc.com/kubernetes-private-docker-registry/>

Toutes les images qui seront "buildé", devront être stocké dans la "registry" (docker push), afin de permettre un déploiement simplifié par la suite dans "kubernetes".

Une fois ces opérations réalisées, vous allez construire les fichiers "manifest" kubernetes nécessaire qui permettrons de déployer complètement votre solution dans votre environnement "minikube".

N'oublier pas les bases de données qui devront faire partie du projet de déploiement "kubernetes" non décrites dans le projet, mais implicite.

Pour cela vous travaillerez ces points,

1. Préparer Minikube
2. Installer le CLI Linkerd+ pré-checks
3. Installer Linkerd(CRDs + control plane) sur Minikube
4. Activer l'injection, développer et déployer une appli maillée (Mesh)
5. Ajouter de l'observabilité (viz), profiles & policies
6. Vérifier, diagnostiquer et exposer les dashboards
7. (En option) mettez en place une stack d'observation (prometheus/Grafana).

Pour ce projet, vous fournirez

- Le schéma d'architecture de votre système.
- L'ensemble des fichiers "dockerfile multiStage".
- L'ensemble des commandes de "buid" et de "push".
- Le ou les fichiers "manifest kubernetes".
- Un « RunBook », sorte de liste de déroulés de commandes permettant de lancer le projet complet (dans un fichier « bash » pourquoi pas!).
- Des captures d'écrans dans un document PDF démontrant le bon déroulement et fonctionnement de votre application déployé dans Kubernetes (Minikube) accompagné du manuel technique et utilisateur.

Livraisons attendues

Bonne nouvelle : vous pouvez utiliser des outils d'IA générative pour vos travaux pratiques. Ces technologies sont puissantes et peuvent vous aider à gagner du temps et à explorer de nouvelles idées. Mais attention, cela ne remplace pas votre réflexion personnelle.

Voici les règles essentielles :

=> **Analyse critique obligatoire**

Les réponses fournies par l'IA ne sont pas toujours exactes. Vous devez vérifier, corriger et valider chaque information en vous appuyant sur ce que nous avons vu en cours et rester dans ce cadre.

=> **Compréhension avant tout**

L'objectif n'est pas d'avoir une solution « prête à l'emploi », mais de comprendre les mécanismes. Si je vous demande d'expliquer votre travail, vous devez être capables de le faire sans l'IA.

=> **Responsabilité**

L'IA est un outil, pas un pilote automatique. Vous restez responsables de la qualité et de la pertinence de votre travail.

En résumé : utilisez l'IA pour aller plus loin, pas pour éviter de réfléchir. C'est votre curiosité et votre esprit critique qui feront la différence.

Bon courage et soyez créatifs... mais toujours rigoureux !

Date limite de remise le : Samedi 2 Mai 2026

Même équipe que le projet précédent, veuillez utiliser le même repository github que précédemment (vous créez un dossier complémentaire correctement identifié pour ce projet).

Remise : Uniquement les attendus décrits ci-dessus via un Git public ou privé(*) adresse du Git qui sera retransmis par mail aux adresses tondeur.herve@yahoo.fr & herve.tondeur@uphf.fr avec les noms des développeurs dans le markdown README qui accompagnera obligatoirement ce projet.

(*) Adresse d'invitation pour le github « tondeur-h » tondeur.herve@yahoo.fr